

RC Implementation Guide (SLT3)

v5.1.0 · Spektrum SLT3 → SR315 → Arduino Nano → Tesla rack · 17 May 2026



1. What v5.1 (SLT3) does

v5.1 is the SLT3 variant of the v5 RC bridge. The plane sticks of the DX8 are replaced by a Spektrum **SLT3 wheel and trigger surface transmitter** bound to a **SR315 dual-protocol surface receiver**. Steering is on the wheel; the trigger fires the P button when pushed forward and held; the AUX rocker selects R / N / D. The Arduino bridge, the Python program, the CAN protocol, and v4.3.3's safety architecture are all unchanged.

Steering math is openpilot's `tools/joystick/joystickd.py` pattern: runtime min/max calibration of the wheel endpoints, a 3% normalized deadband, and a cubic-blend expo curve ($0.4 \cdot x^3 + 0.6 \cdot x$). This is the same shape commaai ships in production for joystick control. Result: small wheel movements give small Tesla-wheel movements (for parking-lot corrections), full wheel rotation reaches full lock-to-lock ($\pm 360^\circ$).

2. Hardware list

Item	Purpose	Source
Spektrum SLT3 transmitter (SPMSLT300)	Operator radio (wheel/trigger)	owned
Spektrum SR315 receiver (SPMSR315)	Receives SLT, emits 3 PWM	owned
Arduino Nano (ATmega328P)	PWM → USB serial bridge	bench supply
3× servo lead wires + 2× jumpers	Rx → Nano	bench supply
Mini-USB cable	Nano → laptop	bench supply
SYS TEC USB-CANmodul1 (model 3204001)	Laptop → car CAN	unchanged from v4

3. Binding the SR315 to the SLT3

SR315 uses a **bind button** (not a bind plug). SLT3 transmits SLT FHSS only; SR315 supports both DSMR and SLT and auto-detects which protocol the transmitter is using. References: SR315 SLT Bind slip sheet (horizonhobby.com/.../SPMSR315-Slip_Sheet.pdf) and SLT3 user guide (horizonhobby.com/.../SPMSLT300-manual-en.pdf).

Step	Action
1	Power the SR315 (any servo port: GND + 5V is enough).
2	Press the SR315 bind button THREE TIMES quickly (within 1.5 s).
3	The receiver LED begins flashing with a pause pattern (bind mode).

- 4 Hold the SLT3 wheel CENTERED, trigger AT REST, AUX1 rocker CENTERED.
- 5 Power on the SLT3. Both LEDs go solid -- bound. Takes ~2 seconds.

Failsafe positions are captured at bind time

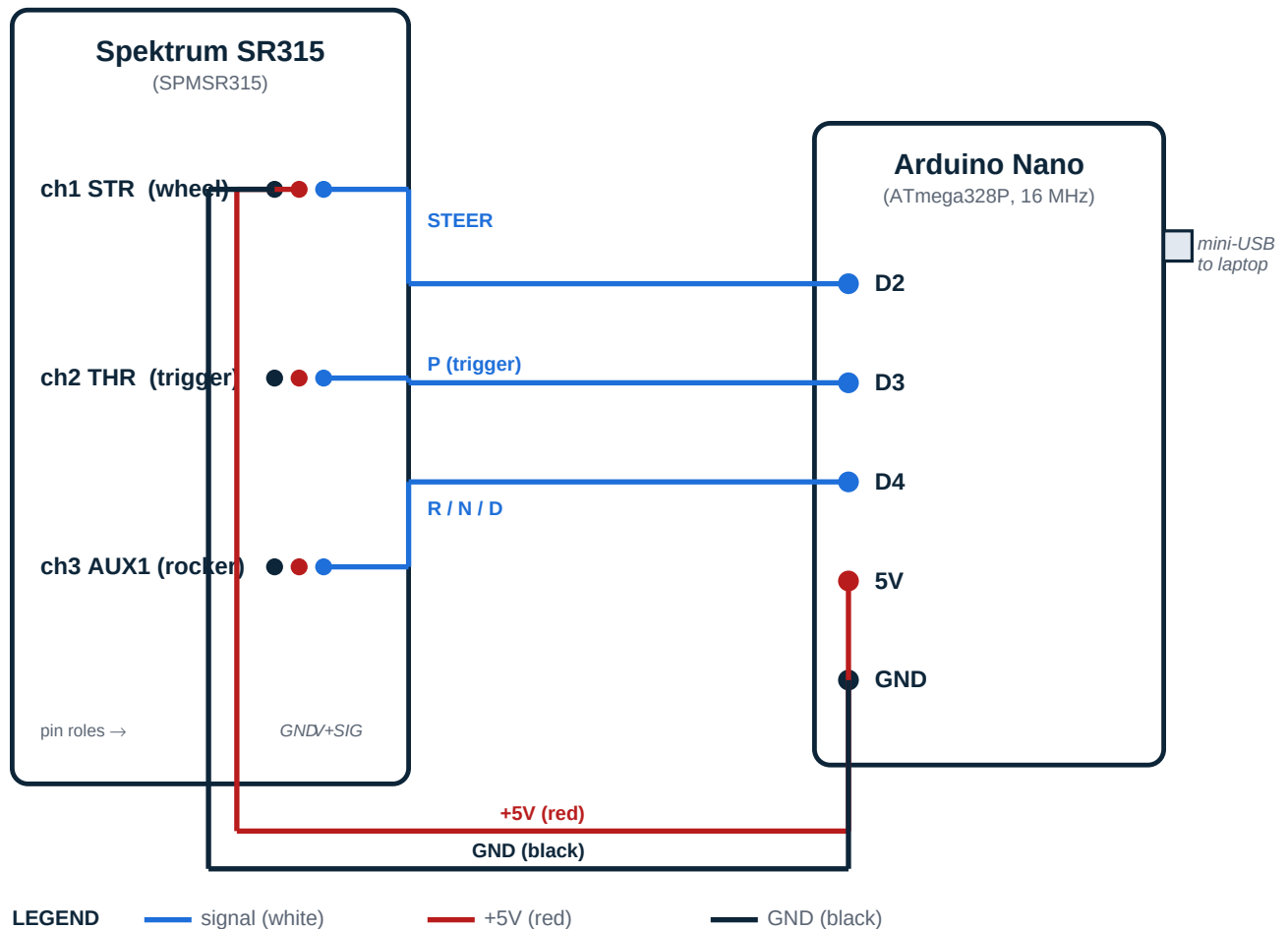
Whatever positions the wheel / trigger / AUX1 are in when the SLT3 powers on become the receiver's failsafe values. Bind with the wheel centered, trigger at rest center, and AUX1 in N. If you later reverse a channel or change the physical orientation of a control, re-bind.

If the LEDs don't go solid

The SR315 may need a firmware update to SLT mode if it was purchased standalone. The one bundled with the SLT3 ships SLT-ready. Re-press the bind button three times, retry. A re-bind never harms either device.

4. Wiring: SR315 → Arduino Nano

All three SR315 channels are wired into the Nano: **ch1 STR** for the wheel, **ch2 THR** for the trigger (P button gesture), and **ch3 AUX1** for the R/N/D rocker. The receiver is powered from the Nano's 5V regulated rail through ch1's positive lead; ch2 and ch3 only need their signal wires connected. SR315's internal rails distribute power across all three servo ports once any one is energized.



5. Pin-by-pin table

SR315 channel & pin	Wire color (servo lead)	Goes to Nano pin	Carries
ch1 STR SIG	white / orange	D2 (PCINT18)	Wheel PWM (steering)
ch1 STR V+	red	5V	Power to Rx
ch1 STR GND	black / brown	GND	Common ground
ch2 THR SIG	white / orange	D3 (PCINT19)	Trigger PWM (P button)
ch3 AUX1 SIG	white / orange	D4 (PCINT20)	Rocker PWM (R/N/D)

ch2 and ch3 share power with ch1

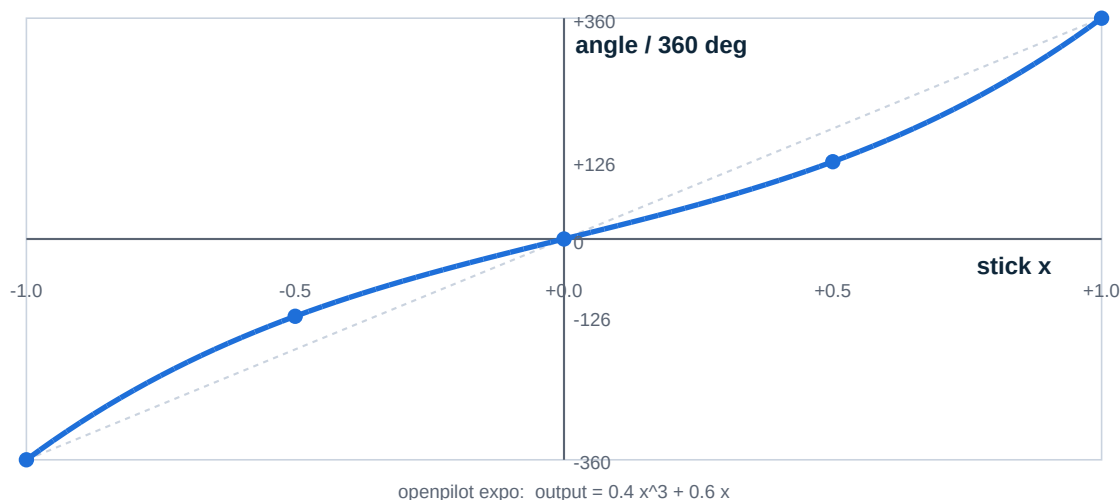
Once ch1 is wired up, SR315's internal rails energize every servo port. ch2 (trigger) and ch3 (AUX1) therefore only need their signal wires connected to the Nano. Tie a single GND between the SR315 and the Nano, not three.

Do not double-power the SR315

If the SR315 is also connected to a separate battery (e.g. the bench BEC), DO NOT connect 5V from the Nano too. Tie grounds together, leave the V+ pin to the Nano disconnected. Otherwise you back-feed two regulators against each other.

6. Steering feel (expo curve)

The stick-to-angle mapping uses openpilot's exact cubic-blend expo from *tools/joystick/joystickd.py*. The shape is **linear-ish near center**, where you make small corrections, and **more aggressive near full deflection**, where you do parking maneuvers. Full stick travel reaches exactly $\pm \text{HARD_ANGLE_LIMIT_DEG}$ ($\pm 360^\circ$ in v4.3.x).



Stick travel → wheel angle, full table

Stick position	Normalized x	Wheel angle
full left	-1.00	-360.0°
75% left	-0.75	-222.7°
50% left	-0.50	-126.0°
25% left	-0.25	-56.2°
10% left	-0.10	-21.7°
5% left	-0.05	-10.8°
centered	+0.00	+0.0°
5% right	+0.05	+10.8°
10% right	+0.10	+21.7°
25% right	+0.25	+56.2°
50% right	+0.50	+126.0°
75% right	+0.75	+222.7°
full right	+1.00	+360.0°

Why expo and not linear

Linear mapping wastes resolution at the center (small stick movements = noticeable wheel jumps) AND makes full lock feel twitchy. Expo gives you fine control where you spend 95% of your driving time, and reserves the last 25% of stick travel for the times you really need to crank the wheel.

7. PRND mapping (AUX1 rocker + trigger)

AUX1 rocker (SR315 ch3) on the SLT3 is a 3-position switch. The program reads the PWM width and selects D / N / R using these hysteresis bands. Rocker LOW selects drive, center is neutral, rocker HIGH selects reverse.

PWM width (us)	AUX1 position	Gear
< 1250	rocker LOW	D (drive)
1250 to 1750	rocker CENTER	N (neutral)
> 1750	rocker HIGH	R (reverse)

A shift is dispatched only when the switch position **changes**: holding the switch in D does not continuously fire shift requests. The existing v4.3.3 non-blocking 200 Hz shift burst handles the SBW_RQ_SCCM transmission unchanged.

Trigger (SR315 ch2) is the spring-return throttle trigger on the SLT3. At rest it sits near center (~1500 us). **Pushing the trigger FORWARD (full brake direction) drops the PWM toward 1000 us and counts as the P button being pressed.** Hold the trigger fully forward for 200 ms to request a shift to P. The trigger springs back to center on release; a 1-second cooldown prevents double-fire.

Trigger PWM (us)	Position	Meaning
<= 1250	full forward (brake)	P button PRESSED
1250 to 1500	deadband	transitioning
> 1500	at rest or pulled	P button RELEASED

8. Flashing the Arduino Nano

Install Arduino CLI from arduino.github.io/arduino-cli/installation, then:

```
cd arduino/tesla_rc_bridge
arduino-cli core install arduino:avr
arduino-cli compile --fqbn arduino:avr:nano:cpu=atmega328 .
arduino-cli upload --fqbn arduino:avr:nano:cpu=atmega328 -p .
```

Many Chinese-clone Nanos ship with the **old bootloader**. If upload fails with avrdude timeout, change the fqbn to *arduino:avr:nano:cpu=atmega328old*.

Finding the serial port

OS	Where to look
Windows	Device Manager → Ports (COM & LPT). 'USB Serial Device (COMx)' or 'CH340'.
macOS	ls /dev/cu.usbserial-* /dev/cu.usbmodem*
Linux	dmesg tail -20 after plug-in. Usually /dev/ttyUSB0 or /dev/ttyACM0.

9. Running the program

Once the Nano is flashed and wired, the SLT3 is bound to the SR315, and the SYS TEC USB-CAN is plugged into both the laptop and the car's chassis CAN tap, run:

```
python tesla_control_rc.py --rc-port COM5
# macOS:
python tesla_control_rc.py --rc-port /dev/cu.usbserial-XXX
```

The GUI is identical to v4.3.3 plus a new **RC INPUT** strip near the top showing the live channel widths, the auto-calibrated stick range, current frame count, and the currently selected gear.

Calibration happens automatically. The first time you sweep the right stick fully left and right, the program records the actual endpoints and uses them from then on. Until that happens, the boot-default 1100..1900 us range is used (which works fine even uncalibrated -- calibration just tightens up the mapping for asymmetric trim).

Standard operating sequence

#	Action
1	Power SLT3, power SR315, plug Nano into laptop.
2	Plug SYS TEC into laptop USB and into the car's chassis CAN tap.
3	python tesla_control_rc.py --rc-port <PORT>
4	In the GUI, click CONNECT. Confirm the bus diagnostic panel populates.
5	Sweep the SLT3 wheel fully left and right once to seed calibration.
6	Verify the RC INPUT panel shows STEER us tracking the wheel.
7	Click ENGAGE. EAC transitions INHIBITED → AVAILABLE → ACTIVE.
8	Steer with the wheel. AUX1 rocker picks D/N/R. Trigger pushed forward and held = P.
9	ESC / Q / E-STOP button / close window = disengage.

Safety carries over from v4.3.3

Hard angle clamp at $\pm 360^\circ$. Rate limit on the worker side at $150^\circ/\text{s}$. RX-timeout watchdog (500 ms with no 0x370 → E-STOP). EAC-bounce watchdog (>5 transitions/s → E-STOP). Real-motion auto-disengage when $DI_vehicleSpeed > 1$ mph. Park-to-engage gate. All of these still apply.

10. Signal-loss detection

Spektrum receivers **hold the last value** on transmitter power-off (SmartSafe). The SR315 keeps emitting whatever PWM widths it last decoded, the Arduino keeps framing them, and the laptop keeps reading them. Without explicit detection, a dead SLT3 looks identical to a stationary wheel. v5 surfaces this in two ways:

Indicator	Trigger	What it means
NO SERIAL	No COBS frame for > 200 ms	Arduino died, USB stalled, or cable unplugged
TX LOST	Aileron PWM unchanged > 2 us for > 3 s	Spektrum holding last value; TX off / out of range
LIVE (green)	Frames arriving, stick changing	Normal operation

The **SIGNAL pill** in the RC INPUT panel shows the current state at all times. By itself the indicator does not interrupt operation -- you decide whether to E-STOP. For unattended or in-car testing, enable the **auto-disengage on signal loss** checkbox in the same panel. When checked, the program drops *ctrl.engaged* on the next UI tick if either SIGNAL condition fires. This is not an E-STOP -- you can re-engage once signal returns -- but it stops the rack from continuing to track a stale target.

Why not auto-disengage by default

On a bench setup with a flaky USB cable or a laptop that stutters its USB hub, the program would re-disengage every few seconds, which is more annoying than helpful. The checkbox lets you flip it on once the bench setup is known-good and you're moving to in-car testing where the consequence of a stuck stick matters more.

TX LOST is a heuristic, not proof

If you legitimately hold the stick perfectly still for 3 seconds, you'll see TX LOST. That's expected -- 3 seconds of perfect stillness is unusual on a real RC operator's stick. If you're driving slow and steady and trip the indicator, raise `RC_FROZEN_STICK_TIMEOUT_S` in the source.

11. Troubleshooting

Symptom	Likely cause	Fix
RC port FAILED to open	Wrong port / driver missing	Re-check Device Manager. CH340 driver if clone Nano.
STEER us stays at 0	White wire not on D2, or SR315 not bound	Rebind SR315 to SLT3, re-check wiring against Section 4
STEER us frozen at 1500	TX off, receiver holds last	Power SLT3 on. Spektrum holds last value on signal loss.
AUX1 wobbles between R and N	Rocker sits near a hysteresis edge	Adjust AUX1 EPA on the SLT3 to push endpoints further out
Shifts spam the log	PWM noise on AUX1	Increase hysteresis bands in tesla_control_rc.py
Wheel travel doesn't reach ± 360 deg	Calibration sweep not done	Sweep the SLT3 wheel fully left, then fully right, once
Wheel jitters at center	Dead band too tight for your wheel	Increase RC_DEADBAND_NORM from 0.03 to 0.05
Wheel won't move past ~ 60 deg	Rack is in MIN_SPEED gating	Enable 30 MPH MODE in the GUI (jacked-up only)
Trigger fires P at rest	Trigger center is below 1250 us at rest	Raise RC_P_PRESS_THRESH_US or adjust SLT3 trigger trim
SIGNAL pill shows TX LOST when wheel is still	Wheel perfectly still > 3 s	Normal heuristic; raise RC_FROZEN_STICK_TIMEOUT_S if too sensitive
SIGNAL pill shows NO SERIAL interconnect	USB cable noise or hub power dip	Plug Nano directly into laptop USB, not through a hub

12. What v5 is NOT

Throttle, brake, and longitudinal control are NOT in v5. The pre-AP 2013 Model S has no CAN-commandable throttle and no iBooster -- adding those is the v6 scope (a pedal interceptor is required). The v5 RC bridge is steering-and-gear only.

v5 also does not replace the keyboard or slider modes. tesla_control.py v4.3.3 still runs exactly as before. v5 is an additional way to drive the same rack, not a replacement.

13. References

openpilot tools/joystick: github.com/commaai/openpilot/blob/master/tools/joystick/joystickd.py

Spektrum SR315 product page: spektrumrc.com/product/sr315-3-channel-dsmr-slt-receiver/SPMSR315.html

Spektrum SR315 SLT bind slip sheet: horizonhobby.com/.../SPMSR315-Slip_Sheet.pdf

Spektrum SLT3 product page (SPMSLT300 bundle):

spektrumrc.com/product/slt3-3-channel-transmitter-with-sr315-receiver/SPMSLT300.html

Spektrum SLT3 user guide: horizonhobby.com/.../SPMSLT300-manual-en.pdf

Consistent Overhead Byte Stuffing (Cheshire & Baker, 1999):

en.wikipedia.org/wiki/Consistent_Overhead_Byte_Stuffing

RCArduino multi-channel PWM read: rcarduino.blogspot.com/2012/04/how-to-read-multiple-rc-channels-draft.html

gregjhogan pre-AP EPAS patch: github.com/gregjhogan/tesla-pre-ap-epas-patch